02-614 Week 11
String Algorithms

Aidan Jan

November 10, 2026

Text Editors

How do text editors store strings? This is an interesting question since when a file is opened in a text editor, it is assumed that the text contained in the file will change... a lot. Previously, we were given a string which we assumed would be constant. However, that is no longer the case, and many of the data structures already covered would not work since we don't have the ability to do preprocessing.

Data Structures

Line-Oriented Data Structures

The file is represented as a linked list with each node containing a linked list.

- This is efficient in inserting lines, since it just means creating a new node.
- However, it is suboptimal for inserting characters since we would have to iterate through the text.
- In the worst case, if the text file is a single, really long line, then this is just equivalent to the naïve way of storing a string.

Gap Buffer

Imagine T as an array, and create a buffer of some size after the cursor.

- If we move the cursor, then copy the character after the buffer to the front. The buffer does not change size with this operation.
- If we type, we begin to fill the buffer.
- The downside is if we fill up the buffer, we must copy the entire string to a new, larger array in order to create a new buffer. Guessing an optimal buffer size becomes important.

This is the strategy Emacs uses.

Piece Tables

Let T be a series of text that is fixed (assumed to not change). Let M be an array (also text) that can grow. Now, we represent the string as a table.

T/M	P	length
T	0	10
M	0	5
T	11	5
M	6	32

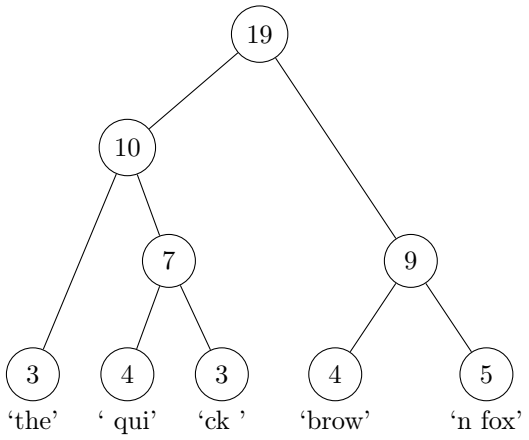
To get a string, we read off of the table. In this example, our string would be composed of characters $T[0:10] + M[0:5] + T[11:16] + M[6:38]$.

- The benefit of this approach is that once we find the correct position in the table, inserting or deleting text happens in constant time.
  - To insert, we split a row into two, the first one the section in T we want to edit, and the second in M, containing our edit.
  - To delete, split the row in T into two rows in T, and just change the starting indices to slice the string.
- When the file is saved, all the changes are written to T.
- The length of the table is never longer than the length of the string. However, M can grow in an unbounded manner if edits are large.

This is the basis of the strategy Code OSS (VS Code) uses. More specifically, Code OSS uses their own designed "piece trees".

Ropes

Consider the the string "the quick brown fox". The Ropes strategy breaks it into a series of nodes (not necessarily separated by word boundaries). One possible example may be [(the), (qui), (ck), (brow), (n fox)]. Now, we make these nodes the leaf nodes of a binary tree.



For the tree, we would use some balanced search tree (Splay, AVL, Red/Black). Nodes store the number of characters represented.

- To reconstruct the string, we read the data in an in-order traversal of the tree.
- To insert text, split the leaf to create a new node, and rebalance the tree.
- To delete text, edit the text in the nodes directly.